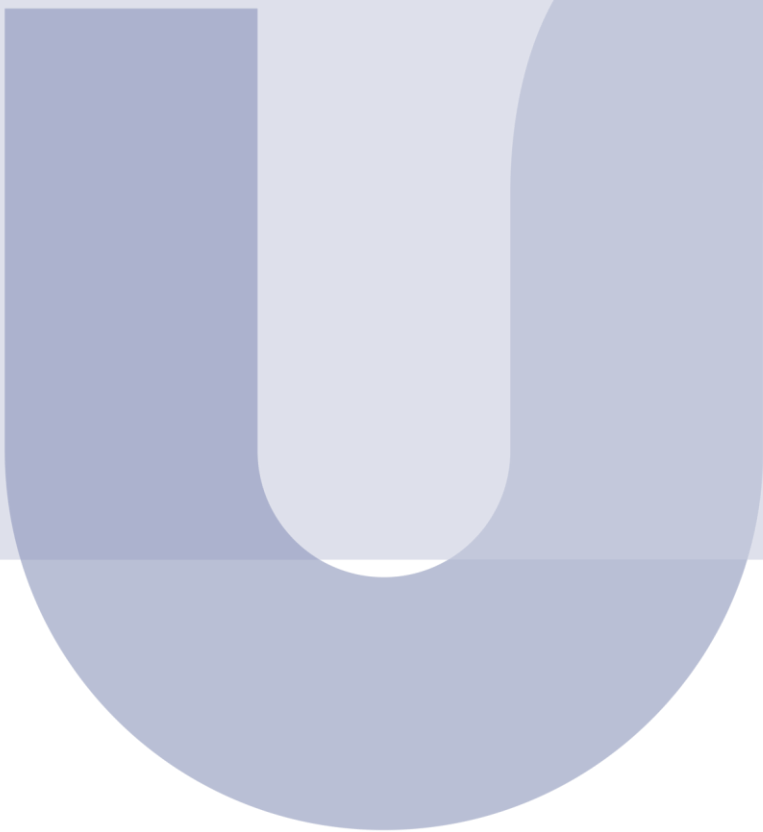


A large, light blue, stylized letter 'U' that serves as a background graphic for the title section. It has rounded corners and a thick stroke.

GUIA DE SQL

Semana 5 – Consultas SQL
Marcos Ricardo Kich

Objetivo do material

Este guia apresenta, de forma progressiva e aplicada, os principais comandos SQL estudados no Atividade Acadêmica. Os exemplos utilizam o cenário da CIA Aérea, o mesmo contexto explorado nas videoaulas.

Tabelas usadas nos exemplos	Papel no cenário
Modelo	Características dos modelos de aeronave, como nome, lugares e autonomia.
Aviao	Aeronaves da companhia e o modelo ao qual pertencem.
Voo	Voos realizados, incluindo partida, destino, avião e piloto.
Piloto	Pilotos cadastrados e suas licenças.
Dependente	Dependentes associados a pilotos.
AptoPilotar	Associação entre pilotos e modelos que estão aptos a pilotar.

Este material não pretende substituir a prática. A melhor forma de aprender SQL é executar, modificar e testar os comandos apresentados.

Apresentação

Nas semanas anteriores, estudamos como identificar requisitos, construir modelos conceituais e transformar essas representações em modelos relacionais. Nesta semana, avançamos para uma nova etapa do desenvolvimento de bancos de dados: a implementação física e a consulta dos dados armazenados.

Para isso, utilizaremos a linguagem SQL (Structured Query Language), padrão amplamente adotado pelos Sistemas Gerenciadores de Banco de Dados (SGBDs). Com SQL, podemos criar tabelas, inserir registros, alterar informações e recuperar dados de acordo com diferentes necessidades de um sistema.

O objetivo deste guia é apresentar os principais conceitos de SQL estudados nesta disciplina, utilizando exemplos baseados no Banco de Dados da CIA Aérea trabalhado nas videoaulas. A proposta é caminhar do simples para o mais elaborado, partindo de consultas básicas e chegando a exemplos com junções, agrupamentos, HAVING e subconsultas.

Como estudar este guia

Leia os exemplos com calma, observe o problema que cada comando tenta resolver e depois teste variações no LiveSQL. Pequenas alterações nos filtros, colunas ou junções ajudam a compreender melhor o funcionamento da linguagem.

O Banco de Dados da CIA Aérea

Ao longo deste guia utilizaremos como referência o Banco de Dados da CIA Aérea, o mesmo cenário explorado nas videoaulas da disciplina.

A Figura 1 apresenta o modelo conceitual utilizado como base para os exemplos. Observe que as informações estão distribuídas entre entidades como Avião, Modelo, Piloto, Voo e Dependente, conectadas por relacionamentos que representam regras do domínio da aplicação.

Embora as consultas SQL sejam executadas sobre tabelas do banco de dados, compreender o modelo conceitual facilita a interpretação das consultas e das relações existentes entre os dados.

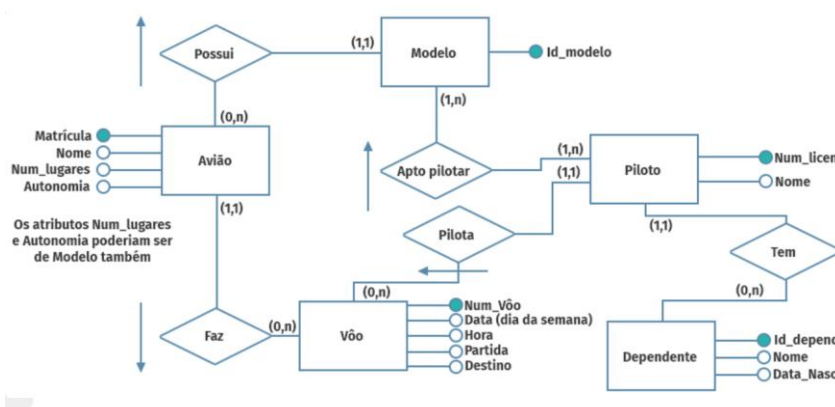


Figura 1 – Modelo conceitual utilizado nos exemplos deste guia

Antes de prosseguir, procure identificar na figura:

- quais entidades serão transformadas em tabelas;
- quais atributos representam informações armazenadas;

- quais relacionamentos conectam as entidades;
- quais consultas poderiam ser realizadas a partir desse modelo.

Nos próximos tópicos veremos como essas estruturas passam a ser acessadas utilizando a linguagem SQL.

1. O que é SQL?

SQL é uma linguagem declarativa. Isso significa que descrevemos o resultado que desejamos obter, e o SGBD é responsável por determinar como executar a operação internamente.

Quando escrevemos um comando SELECT, por exemplo, não explicamos ao banco de dados como percorrer os registros. Apenas indicamos quais dados desejamos visualizar.

```
SELECT *  
FROM Piloto;
```

O comando acima solicita todos os dados cadastrados na tabela Piloto. O asterisco indica que todas as colunas devem ser exibidas.

Quando desejamos visualizar apenas algumas colunas, informamos seus nomes explicitamente.

```
SELECT num_licen, nome  
FROM Piloto;
```

Observe

Em SQL, o mais importante é formular corretamente a pergunta que queremos fazer ao banco de dados. A consulta representa essa pergunta em forma de comando.

2. Recuperando dados com SELECT

O comando SELECT é utilizado para recuperar informações armazenadas em uma ou mais tabelas. Ele é um dos comandos mais importantes da linguagem SQL e será utilizado em praticamente todos os exemplos deste guia.

Para listar todos os modelos de avião cadastrados:

```
SELECT id_modelo,  
       nome,  
       num_lugares,  
       autonomia  
FROM Modelo;
```

Para listar apenas os nomes dos modelos:

```
SELECT nome  
FROM Modelo;
```

Em algumas situações, uma coluna pode apresentar valores repetidos. Para exibir apenas valores distintos, utilizamos DISTINCT.

```
SELECT DISTINCT data  
FROM Voo  
ORDER BY data DESC;
```

Nesse exemplo, são listadas as datas existentes na tabela Voo, sem repetições, ordenadas da mais recente para a mais antiga.

Cláusula	Função
SELECT	Define quais colunas serão exibidas.
FROM	Define de qual tabela os dados serão recuperados.
DISTINCT	Remove repetições no resultado.

3. Filtrando informações com WHERE

Na prática, raramente desejamos visualizar todos os registros de uma tabela. Normalmente queremos responder a perguntas específicas, como: quais aviões são de determinado modelo? Quais voos partiram de uma cidade? Qual piloto realizou certo voo?

A cláusula WHERE permite filtrar os registros que atendem a uma condição.

```
SELECT matricula,  
       id_modelo,
```

```
data_fabricacao
FROM Aviao
WHERE matricula = 'CD3F456';
```

Também podemos utilizar operadores lógicos para combinar condições.

```
SELECT num_voo,
       data,
       hora,
       partida,
       destino,
       matricula,
       num_licen
FROM Voo
WHERE num_licen = 123456
AND partida = 'Porto Alegre';
```

Quando há mais de uma possibilidade para um mesmo critério, podemos utilizar OR.

```
SELECT num_voo,
       data,
       hora,
       partida,
       destino,
       matricula,
       num_licen
FROM Voo
WHERE num_licen = 123456
AND (partida = 'Porto Alegre' OR partida = 'São Paulo');
```

Dica

Use parênteses quando combinar AND e OR. Isso deixa a lógica da consulta mais clara e reduz o risco de obter resultados inesperados.

4. Operadores de busca: IN, NOT IN, BETWEEN e LIKE

SQL oferece operadores que simplificam a escrita de consultas frequentes. Eles tornam o comando mais legível e ajudam a expressar melhor a intenção da busca.

O operador IN permite verificar se um valor pertence a uma lista.

```
SELECT matricula,  
       id_modelo,  
       data_fabricacao  
FROM Aviao  
WHERE id_modelo IN (2, 3);
```

O operador NOT IN realiza a operação inversa, retornando registros que não pertencem à lista informada.

```
SELECT matricula,  
       id_modelo,  
       data_fabricacao  
FROM Aviao  
WHERE id_modelo NOT IN (2, 3);
```

O operador BETWEEN permite consultar valores dentro de um intervalo.

```
SELECT matricula,  
       id_modelo,  
       data_fabricacao  
FROM Aviao  
WHERE id_modelo BETWEEN 2 AND 4;
```

O operador LIKE é utilizado em buscas por padrões de texto. O símbolo % representa qualquer sequência de caracteres.

```
SELECT num_voo,  
       data,  
       hora,  
       partida,  
       destino  
FROM Voo  
WHERE partida LIKE 'São%';
```

Exemplo de interpretação

A condição partida LIKE 'São%' localiza valores que começam com São, como São Paulo. O operador LIKE é muito útil em buscas textuais.

5. Ordenando e limitando resultados

A cláusula ORDER BY organiza os resultados de uma consulta. Por padrão, a ordenação é crescente. Para ordenar em ordem decrescente, utilizamos DESC.

```
SELECT id_modelo,  
       nome,  
       num_lugares,  
       autonomia  
FROM Modelo  
ORDER BY autonomia;  
  
SELECT id_modelo,  
       nome,  
       num_lugares,  
       autonomia  
FROM Modelo  
ORDER BY autonomia DESC;
```

Em alguns casos, desejamos visualizar apenas os primeiros resultados após a ordenação. No Oracle, podemos utilizar FETCH FIRST.

```
SELECT id_modelo,  
       nome,  
       num_lugares,  
       autonomia  
FROM Modelo  
ORDER BY autonomia DESC  
FETCH FIRST 3 ROWS ONLY;
```

Esse comando permite listar os três modelos com maior autonomia, pois primeiro os dados são ordenados de forma decrescente e depois apenas os três primeiros registros são retornados.

Atenção ao SGBD

Alguns comandos variam entre SGBDs. Em SQL Server, por exemplo, é comum encontrar TOP 3 em vez de FETCH FIRST 3 ROWS ONLY.

6. Funções de agregação

As funções de agregação realizam cálculos sobre conjuntos de registros. Elas são muito utilizadas em relatórios, painéis de informação e consultas analíticas.

```
SELECT COUNT(*)  
FROM Aviao  
WHERE id_modelo = 1;
```

A função COUNT conta quantos registros atendem à condição informada.

```
SELECT MAX(autonomia)  
FROM Modelo;  
  
SELECT MIN(num_lugares)  
FROM Modelo;  
  
SELECT SUM(autonomia)  
FROM Modelo;  
  
SELECT AVG(autonomia) AS media_autonomia  
FROM Modelo;
```

Função	Finalidade
COUNT	Conta registros.
MAX	Retorna o maior valor.
MIN	Retorna o menor valor.
SUM	Soma valores.
AVG	Calcula a média.

Essas funções ajudam a transformar dados individuais em informações agregadas, como quantidades, médias e totais.

7. Agrupando informações com GROUP BY

Quando desejamos calcular informações por grupo, utilizamos GROUP BY. Por exemplo, em vez de contar todos os voos da tabela, podemos contar quantos voos partiram de cada cidade.

```
SELECT partida, COUNT(*) AS qtde
FROM Voo
GROUP BY partida;
```

Nesse caso, cada cidade de partida forma um grupo, e COUNT(*) calcula a quantidade de voos em cada grupo.

Quando desejamos filtrar os grupos resultantes, utilizamos HAVING.

```
SELECT partida, COUNT(*) AS qtde
FROM Voo
GROUP BY partida
HAVING COUNT(*) > 1;
```

WHERE ou HAVING?

WHERE filtra registros antes do agrupamento. HAVING filtra grupos depois do GROUP BY. Essa diferença é essencial para construir consultas corretas.

Também podemos utilizar GROUP BY para descobrir o destino com maior quantidade de voos.

```
SELECT destino, COUNT(*) AS qtde
FROM Voo
GROUP BY destino
ORDER BY COUNT(*) DESC
FETCH FIRST 1 ROWS ONLY;
```

8. Relacionando tabelas com JOIN

Em bancos de dados relacionais, as informações normalmente estão distribuídas em várias tabelas. Isso evita redundâncias e melhora a organização dos dados, mas exige que saibamos combinar essas informações quando necessário.

A junção entre tabelas ocorre por meio de atributos relacionados, normalmente chaves primárias e estrangeiras.

Contexto do exemplo

A tabela Aviao possui o atributo id_modelo. Esse atributo permite localizar, na tabela Modelo, as características do modelo de cada aeronave.

Desejamos listar a matrícula do avião, o nome do modelo, o número de lugares e a autonomia.

```
SELECT a.matricula,  
       m.nome,  
       m.num_lugares,  
       m.autonomia  
FROM Aviao a  
INNER JOIN Modelo m  
    ON a.id_modelo = m.id_modelo;
```

Os apelidos a e m tornam a consulta mais curta e facilitam a leitura quando várias tabelas estão envolvidas.

8.1 JOIN com três tabelas

Agora desejamos listar informações de voos, pilotos e aviões. O objetivo é responder: quais voos foram realizados, de onde partiram, para onde foram, quem pilotou e qual aeronave foi utilizada?

```
SELECT v.num_voo,  
       v.partida,  
       v.destino,  
       p.nome AS piloto,  
       a.matricula
```

```

FROM Voo v
INNER JOIN Piloto p
    ON v.num_licen = p.num_licen
INNER JOIN Aviao a
    ON v.matricula = a.matricula;

```

Essa consulta combina três tabelas. A tabela Voo funciona como ponto central, pois se relaciona com Piloto por meio da licença e com Aviao por meio da matrícula.

8.2 JOIN com quatro tabelas

Em consultas mais completas, podemos percorrer várias relações. Suponha que desejamos listar o voo, o piloto, o avião utilizado e o modelo da aeronave.

```

SELECT v.num_voo,
       v.data,
       v.hora,
       v.partida,
       v.destino,
       p.nome AS piloto,
       a.matricula,
       m.nome AS modelo,
       m.num_lugares,
       m.autonomia
FROM Voo v
INNER JOIN Piloto p
    ON v.num_licen = p.num_licen
INNER JOIN Aviao a
    ON v.matricula = a.matricula
INNER JOIN Modelo m
    ON a.id_modelo = m.id_modelo;

```

Esse exemplo mostra uma situação bastante comum em sistemas reais: os dados necessários para responder a uma pergunta estão distribuídos em várias tabelas relacionadas.

Como pensar em JOINS

Antes de escrever a consulta, identifique quais informações deseja exibir e em quais tabelas elas estão. Depois, encontre os atributos que conectam essas tabelas.

8.3 LEFT JOIN

Nem sempre desejamos exibir apenas registros que possuem correspondência em ambas as tabelas. Em alguns casos, queremos manter todos os registros de uma tabela, mesmo quando não existe informação relacionada na outra.

Contexto do exemplo

Desejamos listar todos os pilotos, inclusive aqueles que ainda não possuem dependentes cadastrados.

```
SELECT p.nome AS piloto,
       d.nome AS dependente,
       m.nome AS modelo,
       m.num_lugares,
       m.autonomia
FROM Piloto p
LEFT JOIN Dependente d
       ON p.num_licen = d.num_licen;
```

9. Subconsultas

Uma subconsulta é uma consulta dentro de outra consulta. Ela é útil quando uma parte do problema depende de um resultado intermediário.

Suponha que desejamos descobrir o nome do piloto que possui mais dependentes. O problema pode ser dividido em duas etapas: primeiro descobrir qual licença aparece mais vezes na tabela Dependente; depois buscar o nome do piloto correspondente.

```
SELECT nome
FROM Piloto
WHERE num_licen =
(
    SELECT num_licen
    FROM Dependente
    GROUP BY num_licen
    ORDER BY COUNT(*) DESC
    FETCH FIRST 1 ROWS ONLY
);
```

A consulta interna identifica o número da licença associado à maior quantidade de dependentes. A consulta externa utiliza esse resultado para localizar o piloto.

Leitura da consulta

Uma boa estratégia é ler primeiro a subconsulta. Depois, leia a consulta externa considerando o valor produzido pela consulta interna.

9.1 Subconsulta com NOT IN

Também podemos utilizar subconsultas para localizar registros que não possuem correspondência em outra tabela. Por exemplo: quais modelos não possuem aviões cadastrados?

```
SELECT nome
FROM Modelo
WHERE id_modelo NOT IN
(
    SELECT id_modelo
    FROM Aviao
);
```

A subconsulta retorna os modelos que aparecem na tabela Aviao. A consulta externa lista os modelos que não estão nesse conjunto.

10. Consultas mais elaboradas

À medida que avançamos no estudo de SQL, as consultas passam a combinar diferentes recursos. Isso é natural, pois problemas reais geralmente exigem filtros, junções, agrupamentos e cálculos ao mesmo tempo.

10.1 Quantidade de voos por piloto

Queremos listar cada piloto e a quantidade de voos em que ele está alocado. Para isso, precisamos relacionar Piloto e Voo, agrupar pelo piloto e contar os voos.

```
SELECT p.nome,
       COUNT(*) AS qtde_voos
FROM Piloto p
INNER JOIN Voo v
```

```
ON p.num_licen = v.num_licen
GROUP BY p.nome;
```

Esse exemplo já combina JOIN, GROUP BY e COUNT.

10.2 Modelo de avião que participou de mais voos

Agora queremos descobrir qual modelo de avião aparece em mais voos. A informação do voo está na tabela Voo, a matrícula está em Aviao e o nome do modelo está em Modelo.

```
SELECT m.nome AS modelo,
       COUNT(*) AS qtde_voos
FROM Voo v
INNER JOIN Aviao a
    ON v.matricula = a.matricula
INNER JOIN Modelo m
    ON a.id_modelo = m.id_modelo
GROUP BY m.nome
ORDER BY COUNT(*) DESC
FETCH FIRST 1 ROWS ONLY;
```

O resultado mostra o modelo mais utilizado nos voos cadastrados.

10.3 Dois modelos mais utilizados nos voos

Podemos adaptar a consulta anterior para exibir os dois modelos mais utilizados.

```
SELECT m.nome AS modelo,
       COUNT(*) AS qtde_voos
FROM Voo v
INNER JOIN Aviao a
    ON v.matricula = a.matricula
INNER JOIN Modelo m
    ON a.id_modelo = m.id_modelo
GROUP BY m.nome
ORDER BY COUNT(*) DESC
FETCH FIRST 2 ROWS ONLY;
```

Esse tipo de consulta é comum quando desejamos gerar rankings ou relatórios de uso.

10.4 HAVING com subconsulta

Um exemplo mais avançado ocorre quando o filtro aplicado ao grupo depende de outro cálculo. Suponha que desejamos listar os pilotos que realizaram mais voos do que a média de voos dos pilotos que já realizaram pelo menos um voo.

```

SELECT p.nome,
       COUNT(*) AS qtde_voos
FROM Piloto p
INNER JOIN Voo v
       ON p.num_licen = v.num_licen
GROUP BY p.nome
HAVING COUNT(*) >
(
  SELECT AVG(qtde)
  FROM
  (
    SELECT COUNT(*) AS qtde
    FROM Voo
    GROUP BY num_licen
  )
);

```

Essa consulta combina JOIN, GROUP BY, HAVING, subconsulta e AVG. Primeiro, a subconsulta interna calcula a quantidade de voos por piloto. Depois, a subconsulta externa calcula a média dessas quantidades. Por fim, o HAVING compara a quantidade de voos de cada piloto com essa média.

Por que esse exemplo é importante?

Ele mostra que SQL permite resolver problemas em etapas lógicas. Mesmo consultas longas podem ser compreendidas quando dividimos o raciocínio em partes menores.

10.5 Dependentes do piloto com mais dependentes

Também podemos utilizar uma subconsulta para encontrar o piloto com mais dependentes e, em seguida, listar os nomes dos seus dependentes.

```

SELECT d.nome
FROM Dependente d
WHERE d.num_licen =
(
  SELECT num_licen
  FROM Dependente
  GROUP BY num_licen
  ORDER BY COUNT(*) DESC
  FETCH FIRST 1 ROWS ONLY
);

```


Observe que a tabela Dependente aparece tanto na consulta externa quanto na subconsulta. Na consulta externa, ela fornece os nomes dos dependentes; na subconsulta, ela é usada para descobrir qual piloto possui mais dependentes.

11. SQL e Álgebra Relacional

A SQL possui forte relação com os conceitos estudados anteriormente em Álgebra Relacional. Essa relação ajuda a compreender que a linguagem SQL não surgiu isoladamente, mas está apoiada em fundamentos formais dos bancos de dados relacionais.

Álgebra Relacional	SQL	Ideia principal
Seleção	WHERE	Filtrar linhas.
Projeção	SELECT	Escolher colunas.
Junção	JOIN	Relacionar tabelas.
Agrupamento	GROUP BY	Organizar registros em grupos.
Filtro sobre grupos	HAVING	Filtrar resultados agregados.

Por exemplo, uma seleção em Álgebra Relacional pode ser representada em SQL por uma cláusula WHERE.

```
SELECT nome,  
       num_lugares  
FROM Modelo  
WHERE num_lugares > 150;
```

Uma projeção, por sua vez, corresponde à escolha das colunas que serão exibidas.

```
SELECT nome  
FROM Piloto;
```

Uma junção pode ser expressa com JOIN.

```
SELECT a.matricula, m.nome  
FROM Aviao a
```

```
INNER JOIN Modelo m
ON a.id_modelo = m.id_modelo;
```

12. Resumo dos principais comandos

Comando ou cláusula	Finalidade
SELECT	Recuperar dados.
FROM	Indicar a tabela de origem.
WHERE	Filtrar registros.
DISTINCT	Remover valores repetidos.
ORDER BY	Ordenar resultados.
FETCH FIRST	Limitar quantidade de linhas retornadas.
COUNT	Contar registros.
AVG	Calcular média.
MAX / MIN	Maior e menor valor.
SUM	Somar valores.
GROUP BY	Agrupar registros.
HAVING	Filtrar grupos.
INNER JOIN	Relacionar registros correspondentes entre tabelas.
LEFT JOIN	Manter registros da tabela à esquerda, mesmo sem correspondência.
IN / NOT IN	Comparar com listas ou resultados de subconsultas.
LIKE	Buscar padrões textuais.
BETWEEN	Consultar intervalos.

Síntese da semana

Nesta semana, SQL é estudada como instrumento para transformar o modelo relacional em prática: criar estruturas, manipular dados e recuperar informações relevantes por meio de consultas.

Considerações finais

Aprender SQL exige prática constante. A leitura dos comandos é importante, mas a compreensão se consolida quando você executa as consultas, observa os resultados e faz pequenas alterações para testar novas hipóteses.

Utilize os exemplos deste guia como ponto de partida. Experimente alterar filtros, incluir novas colunas, modificar ordenações e construir novas perguntas a partir do cenário da CIA Aérea.

Ao dominar os comandos apresentados, você terá uma base sólida para compreender consultas SQL mais elaboradas e para avançar no desenvolvimento de aplicações que utilizam bancos de dados relacionais.